

Chapter 6: Functions - Complete Notes (Sections A-L)

- A. Function Declarations vs Function Expressions
- B. Parameters, Default Values, Rest Parameters, arguments
- C. Arrow Functions vs Regular Functions (Syntax + this)
- D. Return Values - Implicit vs Explicit Return
- E. First-Class Functions (Functions as Values)
- F. Higher-Order Functions (Functions That Take/Return Functions)
- G. Closures and Lexical Scoping
- H. IIFEs (Immediately Invoked Function Expressions)
- I. this Keyword Deep Dive (call, apply, bind)
- J. Callback Functions
- K. Pure Functions & Side Effects
- L. Recursion

A. Function Declarations vs Function Expressions

1. Function Declaration

```
function greet(name) {  
  return `Hello, ${name}`;  
}
```

Starts with the function keyword, has a name, no semicolon needed.

2. Function Expression

```
const greet = function (name) {  
  return `Hello, ${name}`;  
};
```

A function assigned to a variable. Semicolon needed. Can be named or anonymous.

3. Key difference: hoisting

Feature	Declaration	Expression
Hoisted?	Yes, fully	Only the variable (if var), not the function
Callable before its line?	Yes	No

```
sayHi(); // works  
function sayHi() { console.log("hi"); }  
  
sayBye(); // TypeError  
var sayBye = function () { console.log("bye"); };  
  
sayHello(); // ReferenceError (TDZ)  
const sayHello = function () { console.log("hello"); };
```

4. Why it matters

Declarations let you organize helper functions below their usage. Expressions are required for conditional assignment:

```
let operation;
if (useMetric) {
  operation = function (km) { return km; };
} else {
  operation = function (km) { return km * 0.62; };
}
```

5. Quick identification trick

Does function start the statement with nothing before it? Then it's a declaration. Otherwise, expression.

B. Parameters, Default Values, Rest Parameters, arguments

1. Basic parameters

Missing arguments become undefined. Extra unnamed arguments are ignored unless captured.

2. Default parameters

```
function greet(name, greeting = "Hello") {
  return `${greeting}, ${name}`;
}
greet("Aniket");           // "Hello, Aniket"
greet("Aniket", undefined); // "Hello, Aniket" - undefined triggers
default
greet("Aniket", null);      // "null, Aniket" - null does NOT
trigger default
```

Only undefined triggers the default. Defaults can reference earlier parameters.

3. Rest parameters (...)

```
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3, 4); // 10
```

Collects remaining arguments into a **real array**. Must be the last parameter.

4. The arguments object

Array-like (not a true array), available only in regular functions, not arrow functions. Rest params are the modern replacement - a real array with all array methods.

	arguments	Rest params
Type	Array-like	Real array
In arrow functions?	No	Yes
Array methods directly?	No	Yes

C. Arrow Functions vs Regular Functions (Syntax + this)

1. Syntax

```
function multiply(a, b) { return a * b; }
const multiply2 = (a, b) => a * b;
const square = n => n * n;           // single param, no parens
needed
const sayHi = () => console.log("hi"); // no params, empty parens
required
```

2. The core this rule

- **Regular function:** this = whatever is written immediately before the dot, **at the call**. Dynamic.
- **Arrow function:** no this of its own; borrows this from the nearest surrounding **regular function** (or top-level if none). Lexical, fixed at write time.

3. Regular function: this changes with the call

```
const person = {
  name: "Aniket",
  greet: function () { console.log(this.name); }
};
person.greet();           // "Aniket" - this = person

const detached = person.greet;
detached();               // undefined - nothing before the dot
```

4. Arrow function: this is frozen at write time

```
const person = {
  name: "Aniket",
  greet: () => { console.log(this.name); }
};
person.greet(); // undefined - arrow borrows top-level this, not
person
```

An object { } is NOT a function and gives no this. The arrow looks past it to the top level.

5. Arrow works correctly when written inside a regular function

```
const person = {
  name: "Aniket",
  start: function () {
    const inner = () => console.log(this.name);
    inner();
  }
};
person.start(); // "Aniket" - inner borrows start's this, which is
person
```

6. The setTimeout problem - why arrows exist

```
// Breaks: regular function inside loses this
const timer = {
  seconds: 0,
  start: function () {
    setTimeout(function () { this.seconds++; }, 1000); // this is
wrong here
  }
};

// Fixed: arrow borrows start's this
const timer2 = {
  seconds: 0,
  start: function () {
    setTimeout(() => { this.seconds++; }, 1000); // this = timer2
  }
};
```

7. Other differences

Feature	Regular	Arrow
Own this	Yes	No
arguments object	Yes	No
Usable with new	Yes	No
.call/.apply/.bind change this	Yes	No (ignored)
Implicit return	No	Yes

8. When to use which

Regular functions for object methods that need this; arrow functions for callbacks inside methods, and short one-off callbacks like `arr.map(n => n * 2)`.

D. Return Values - Implicit vs Explicit Return

1. What return does

Sends a value back to the caller and **immediately stops** the function. Code after return is unreachable.

2. No return → undefined

`console.log` only prints; it does not return anything. A function with no return still runs fully, but gives back undefined.

3. Explicit return

Required whenever the function body uses `{ }` - all regular functions, and arrow functions with braces.

```
function square(n) { return n * n; }
const square2 = (n) => { return n * n; };
```

4. Implicit return (arrow functions only)

```
const square3 = n => n * n; // no braces, automatically returned
const double = n => { n * 2; }; // BUG: braces added, no return → undefined
```

5. Returning an object - the parentheses trap

```
const makeUser = (name) => { name: name }; // WRONG: read as a function body block
const makeUser2 = (name) => ({ name: name }); // CORRECT: parens mark it as an expression
```

6. Returning early (multiple returns)

```
function checkAge(age) {
  if (age < 0) return "Invalid age";
  if (age < 18) return "Minor";
  return "Adult";
}
```

Only the first return reached runs.

7. return with no value

`return;` alone exits early and gives back undefined - used for guard clauses.

E. First-Class Functions (Functions as Values)

1. What “first-class” means

A function is treated like any other value: it can be stored in a variable, stored in an object/array, passed as an argument, and returned from a function.

2-3. Stored in a variable / object / array

```
const greet = function () { console.log("Hi!"); };

const obj = { sayHi: function () { console.log("Hi!"); } };

const funcList = [function () { console.log("one"); }];
funcList[0]();
```

4. Passed as an argument

```
function callTwice(fn) { fn(); fn(); }
function sayHello() { console.log("Hello!"); }
callTwice(sayHello); // pass the function itself, NOT sayHello()
```

Meaning	
sayHello	the function itself, a value, not run
sayHello()	runs it immediately, passes its <i>result</i>

5. Returned from a function

```
function makeMultiplier(factor) {
  return function (n) { return n * factor; };
}
const double = makeMultiplier(2);
double(5); // 10
```

(This is an early look at closures - Section G.)

6. Why it matters

Enables two patterns: **higher-order functions** (Section F) and **callbacks** (Section J). Array methods, setTimeout, event listeners all depend on functions being first-class.

F. Higher-Order Functions (Functions That Take/Return Functions)

1. Definition

A function that takes another function as an argument, returns a function, or both.

2. Type 1 - Takes a function in

```
function greetUser(name, formatter) {
  console.log(formatter(name));
}
function upperCase(str) { return str.toUpperCase(); }
greetUser("Aniket", upperCase); // "ANIKET"
```

.map(), .filter(), .forEach(), addEventListener() all work this way.

3. Type 2 - Returns a function

```
function makeGreeter(greeting) {
  return function (name) { console.log(`${greeting}, ${name}`); };
}
const sayHello = makeGreeter("Hello");
sayHello("Aniket"); // "Hello, Aniket"
```

4. Both at once

```
function withLogging(fn) {
  return function (...args) {
    console.log("calling function...");
    return fn(...args);
  };
}
const loggedAdd = withLogging((a, b) => a + b);
loggedAdd(2, 3); // logs, then returns 5
```

5. Why it matters

Reusability, avoids repeated code, composability. Nearly every array method and `setTimeout` is a higher-order function.

G. Closures and Lexical Scoping

1. Lexical scoping (recap)

A function's access to variables is decided by **where it is written**, not how it's called.

2. What a closure is

A closure is when a function **remembers** variables from the scope it was created in, even after that outer scope has finished running.

```
function outer() {
  const secret = "I am hidden";
  return function inner() { console.log(secret); };
}
const myFunc = outer();
myFunc(); // "I am hidden" - secret is remembered
```

3. Why the variable survives

Normally local variables are cleaned up after a function finishes. If an inner function still references them, JavaScript keeps them alive just for that inner function.

4. The counter example

```
function makeCounter() {
  let count = 0;
  return function () { count++; return count; };
}
const counter1 = makeCounter();
counter1(); // 1
```

```

counter1(); // 2

const counter2 = makeCounter();
counter2(); // 1 - separate, independent count

```

Each call to the outer function creates a brand-new closure; counters don't share state.

5. Data privacy via closures (module pattern)

```

function createBankAccount(initialBalance) {
  let balance = initialBalance;
  return {
    deposit: (amount) => balance += amount,
    withdraw: (amount) => balance -= amount,
    checkBalance: () => balance
  };
}

```

balance cannot be accessed directly from outside.

6. The classic var loop bug

```

function createFunctions() {
  var funcs = [];
  for (var i = 0; i < 3; i++) {
    funcs.push(function () { console.log(i); });
  }
  return funcs;
}
// all three print 3 - var is function-scoped, one shared i

// Fix: use let
for (let i = 0; i < 3; i++) {
  funcs.push(function () { console.log(i); }); // each gets its own i
}

```

H. IIFEs (Immediately Invoked Function Expressions)

1. What an IIFE is

A function that runs immediately upon definition, never called separately.

```

(function () {
  console.log("I ran immediately!");
})();

```

2. Syntax breakdown

Wrapping parentheses turn the function into an expression; the trailing () calls it immediately. Without the wrapping parens, JavaScript reads function as the start of a declaration and throws a syntax error.

3. Arrow IIFE

```
((() => { console.log("arrow IIFE ran!"); })());
```

4. Why it exists - avoiding global scope pollution

```
(function () {  
  var count = 0;  
  console.log(count); // 1  
})();  
console.log(count); // ReferenceError - count is private to the IIFE
```

5. Historical context

Before let/const block scope and ES Modules, IIFEs were the only way to create a private scope with var. Still seen in older libraries.

6. IIFE + closure

```
const counter = (function () {  
  let count = 0;  
  return function () { count++; return count; };  
})();  
counter(); // 1  
counter(); // 2
```

7. Less essential today

let/const and ES Modules cover most of what IIFEs used to be needed for, but they're still used for one-time setup code and understanding older codebases.

I. this Keyword Deep Dive (call, apply, bind)

1. Why manual control is needed

```
function introduce() { console.log(`Hi, I am ${this.name}`); }  
const person = { name: "Aniket" };  
introduce(); // this is not person
```

2. .call() - run now, args listed individually

```
introduce.call(person); // "Hi, I am Aniket"  
function introduce2(city, hobby) {  
  console.log(`Hi, I am ${this.name} from ${city}, I like  
  ${hobby}`);  
}  
introduce2.call(person, "Nagpur", "coding");
```

3. .apply() - run now, args as an array

```
introduce2.apply(person, ["Nagpur", "coding"]);
```

Memory trick: "Apply takes an Array."

4. .bind() - doesn't run now, returns a new function

```
const boundIntroduce = introduce.bind(person);
boundIntroduce(); // "Hi, I am Aniket" - this is permanently locked
to person
```

Once bound, nothing can override this again - not even calling it as another object's method.

5. Comparison

	Runs immediately?	Returns	Args format
.call()	Yes	the function's return value	individually
.apply()	Yes	the function's return value	array
.bind()	No	a new function	individually

6. Fixing lost this with .bind()

```
const timer = {
  seconds: 0,
  start: function () {
    setTimeout(function () {
      this.seconds++;
    }.bind(this), 1000); // locks this to timer
  }
};
```

Arrow functions are the more common modern fix for this exact problem.

7. Partial application with .bind()

```
function multiply(a, b) { return a * b; }
const double = multiply.bind(null, 2); // locks a = 2
double(5); // 10
```

8. Arrow functions ignore all three

.call(), .apply(), .bind() have no effect on arrow functions - they never had their own this to override.

J. Callback Functions

1. Definition

A function passed into another function, to be called later.

```
function processUser(name, callback) { callback(name); }
processUser("Aniket", function (name) { console.log(`Hi, ${name}`);
});
```

2. Synchronous vs asynchronous callbacks

```
// Synchronous - runs immediately, in order
[1, 2, 3].forEach(n => console.log(n));
console.log("done");
// 1 2 3 done

// Asynchronous - runs later
console.log("start");
setTimeout(() => console.log("callback ran"), 1000);
console.log("end");
// start, end, callback ran (after ~1s)
```

3. Named vs anonymous vs arrow callbacks

All three forms are valid and common; inline anonymous/arrow callbacks dominate modern code since the function is usually only needed once.

4. Extra arguments from array methods

```
[10, 20, 30].forEach((value, index, array) => console.log(value,
index, array));
```

.forEach/.map/.filter always call the callback with (currentValue, index, wholeArray).

5. Error-first callbacks

Older Node.js convention: first argument reserved for an error, null if none. Largely replaced by Promises today.

6. Callback hell

```
getUser(id, function (user) {
  getPosts(user, function (posts) {
    getComments(posts, function (comments) {
      console.log(comments);
    });
  });
});
```

Deeply nested async callbacks are hard to read/debug - the motivation for Promises, covered later.

7. Callbacks you already use

.map, .filter, .reduce, setTimeout, addEventListener - the function passed into any of these is a callback. Higher-order functions and callbacks describe the same relationship from two sides.

K. Pure Functions & Side Effects

1. What a side effect is

Anything a function does that reaches outside its own scope: modifying outer variables, mutating arguments, console.log, network requests, DOM changes.

2. What a pure function is

1. Same input → always the same output.
2. No side effects.

```
function add(a, b) { return a + b; } // pure
```

3. Impure - depends on outside state

```
let taxRate = 0.18;
function calculateTax(amount) { return amount * taxRate; } // impure

// Fixed:
function calculateTax2(amount, rate) { return amount * rate; } //
pure
```

4. Impure - mutates its input

```
function addItem(cart, item) { cart.push(item); return cart; } //
impure, mutates original

// Fixed:
function addItem2(cart, item) { return [...cart, item]; } // pure,
returns new array
```

5. console.log is technically impure

Accepted in practice as an allowed exception for debugging; purity mainly matters for core logic.

6. Why pure functions matter

Predictable, easy to test, safe to reuse, easier to debug, safe for async/parallel code.

7. The core habit: return new data, don't mutate

```
// Impure
function updateAge(person, newAge) { person.age = newAge; return
person; }
```

```
// Pure
function updateAge2(person, newAge) { return { ...person, age:
newAge }; }
```

8. Array methods: pure vs impure

Method	Mutates original?	Pure?
.map()	No	Pure
.filter()	No	Pure
.reduce()	No	Pure
.push()	Yes	Impure
.pop()	Yes	Impure
.sort()	Yes	Impure
.splice()	Yes	Impure

9. Purity doesn't mean zero side effects in a program

Real programs need side effects somewhere (DOM updates, saving data). The goal is isolating side effects to a small part of the code while keeping core logic pure.

L. Recursion

1. What recursion is

A function that calls itself to solve smaller versions of the same problem.

```
function countdown(n) {
  console.log(n);
  if (n === 0) return;
  countdown(n - 1);
}
```

2. Two required parts

- **Base case:** stops the recursion.
- **Recursive case:** calls itself, moving closer to the base case.

Missing the base case causes infinite recursion and a stack overflow.

3. Without a base case

```
function brokenCountdown(n) {
  console.log(n);
  brokenCountdown(n - 1); // never stops
}
// RangeError: Maximum call stack size exceeded
```

4. Classic example: factorial

```
function factorial(n) {
  if (n === 1) return 1; // base case
  return n * factorial(n - 1); // recursive case
}
factorial(5); // 120
```

5. Tracing the call stack

```
factorial(3) -> 3 * factorial(2)
  factorial(2) -> 2 * factorial(1)
    factorial(1) -> base case, returns 1
  factorial(2) returns 2 * 1 = 2
factorial(3) returns 3 * 2 = 6
```

Each call pauses, waiting for the inner call, then “unwinds” back up once the base case resolves.

6. Summing an array recursively

```
function sumArray(arr) {
  if (arr.length === 0) return 0;
  return arr[0] + sumArray(arr.slice(1));
}
sumArray([1, 2, 3, 4]); // 10
```

7. Recursion vs a loop

	Loop	Recursion
Simple repetition	Usually easier to read	Not always
Naturally nested/branching data (trees, nested objects)	Harder	Much more natural
Extra memory per step	No	Yes (call stack)
Risk	Infinite loop	Stack overflow

8. Where recursion shines: nested data

```
function countNodes(node) {
  let count = 1;
  for (const child of node.children) {
    count += countNodes(child);
  }
  return count;
}
```

Naturally handles unknown depth of nesting - hard to express cleanly with a loop alone.

9. Stack overflow - the common bug

Always check: does every recursive call move measurably closer to the base case? If not, it will eventually crash.